

Actividad Guiada 1 de Algoritmos de Optimizacion

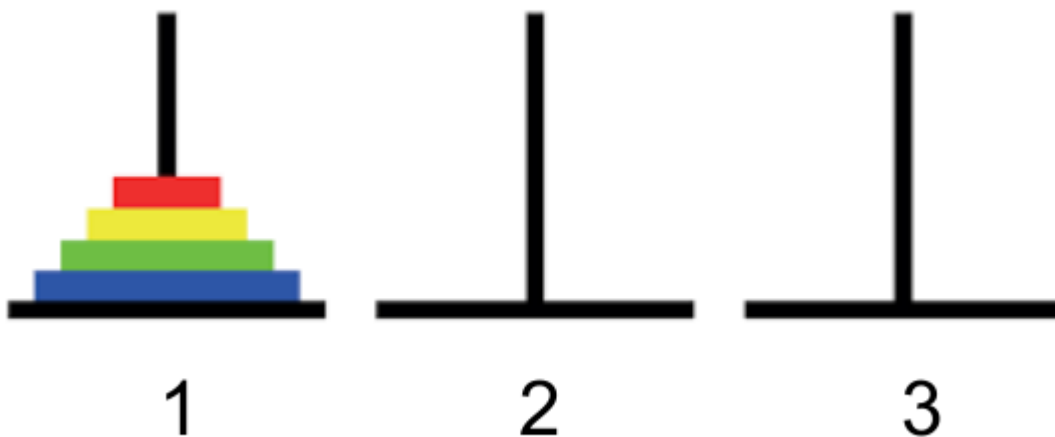
Nombre: Raul Reyero(Pon aqui tu nombre)

https://colab.research.google.com/drive/1mhqzKQS9M-l865_pnNbrv7EIP9JVVcP-?usp=sharing (Pon aquí tu cuaderno)

<https://github.com/rubengp02/03MIAR-Algoritmos-de-Optimizacion> (Pon aquí tu proyecto git)

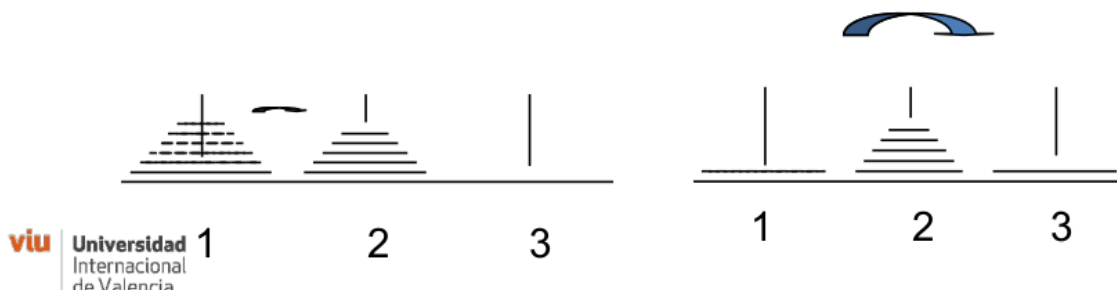
1. Elemento de lista
2. Elemento de lista

✓ Torres de Hanoi - Divide y venceras



Resolver(Total_fichas=4, Desde=1, Hasta=3) es valido con:

- Resolver(Total_fichas=3, Desde=1, Hasta=2)
- Mover(Desde=1, Hasta=3)
- Resolver(Total_fichas=3, Desde=2, Hasta=3)



```
#Torres de Hanoi - Divide y venceras
```

```
#####
```

```
#####
```

```
def Torres_Hanoi(N, desde, hasta):
```

```
    #N - Nº de fichas
```

```
    #desde - torre inicial
```

```
    #hasta - torre fina
```

```
    if N==1 :
```

```
        print("Lleva la ficha desde " + str(desde) + " hasta " + str(hasta))
```

```
    else:
```

```
        Torres_Hanoi(N-1, desde, 6-desde-hasta) # Hasta será igual a 2 (la torre
```

```
        print("Lleva la ficha desde " + str(desde) + " hasta " + str(hasta))
```

```
        Torres_Hanoi(N-1, 6-desde-hasta, hasta)
```

```
Torres_Hanoi(5, 1, 3)
```

```
#####
```

```
Lleva la ficha desde 1 hasta 3
```

```
Lleva la ficha desde 1 hasta 2
```

```
Lleva la ficha desde 3 hasta 2
```

```
Lleva la ficha desde 1 hasta 3
```

```
Lleva la ficha desde 2 hasta 1
```

```
Lleva la ficha desde 2 hasta 3
```

```
Lleva la ficha desde 1 hasta 3
```

```
Lleva la ficha desde 1 hasta 2
```

```
Lleva la ficha desde 3 hasta 2
```

```
Lleva la ficha desde 3 hasta 1
```

```
Lleva la ficha desde 2 hasta 1
```

```
Lleva la ficha desde 3 hasta 2
```

```
Lleva la ficha desde 1 hasta 3
```

```
Lleva la ficha desde 1 hasta 2
```

```
Lleva la ficha desde 3 hasta 2
```

```
Lleva la ficha desde 1 hasta 3
```

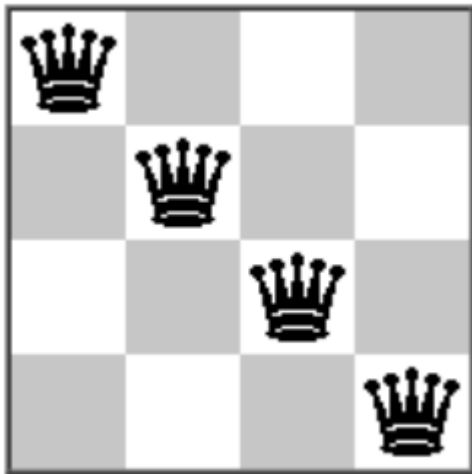
```
Lleva la ficha desde 2 hasta 1
```

```
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 3 hasta 1
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
```

› Cambio de monedas - Técnica voraz

↳ 2 celdas ocultas

✓ N Reinas - Vuelta Atrás(Backtracking)



```
#N Reinas - Vuelta Atrás()
#####

#Verifica que en la solución parcial no hay amenazas entre reinas
#####
# Comprueba si la solución parcial sigue siendo válida tras colocar
# una reina en la fila 'etapa'
def es_prometedora(SOLUCION, etapa):

    # Columna en la que se acaba de colocar la reina
    col = SOLUCION[etapa]

    # Se compara únicamente la reina recién colocada (fila = etapa) con las
```

```

# Antes se comparaban todas las parejas de reinas ( $O(n^2)$ ) ahora solo se
for fila in range(etapa):

    # El uso de count() dentro de un bucle hacía que el coste fuera muy
    if SOLUCION[fila] == col:
        return False

    # El bucle anterior comparaba todas las reinas, con un coste de  $O(n^2)$ 
    # Coste total de esta comprobación es de  $O(n)$ 
    if abs(etapa - fila) == abs(col - SOLUCION[fila]):
        return False

# Antes  $O(n^2)$ , ahora  $O(n)$ 
return True

#Traduce la solución al tablero
#####
def escribe_solucion(S):
#####
    n = len(S)
    for x in range(n):
        print("")
        for i in range(n):
            if S[i] == x+1:
                print(" X ", end="")
            else:
                print(" - ", end="")

#Proceso principal de N-Reinas
#####
def reinas(N, solucion=[],etapa=0):
#####
    ### ....
    if len(solucion) == 0:          # [0,0,0...]
        solucion = [0 for i in range(N) ]

    for i in range(1, N+1):
        solucion[etapa] = i
        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

    solucion[etapa] = 0

reinas(4,solucion=[],etapa=0)

```

```

[2, 4, 1, 3]
[3, 1, 4, 2]

```

```
escribe_solucion([3, 1, 4, 2])
```

```
- X - -  
- - - X  
X - - -  
- - X -
```

Empieza a programar o a [crear código](#) con IA.